C++14 Language and Library Enhancements

Theory, Edge Cases & Code Examples

- ✓ Return Type Deduction for Normal Functions
- ✓ Generic Lambdas
- ✓ Lambda Init-Capture (Move Capture)
- ✓ decltype(auto) for Exact Type Deduction
- ✓ Relaxed constexpr Functions

→ *Swipe to learn* →

C++14 Language and Library Enhancements

Theory

📌 Return Type Deduction for Normal Functions

In C++11, `auto` return type deduction only worked for lambdas. C++14 extends this to **any function**. The compiler deduces the return type from the return statement(s).

```
auto square(int x) {
    return x * x;  // Deduces return type as int
}
```

This works when all return statements deduce to the same type. If different return paths produce different types, compilation fails:

```
auto ambiguous(bool b) {
    if (b) return 1;      // int
    else return 1.5;      // double → ERROR: mismatched deduction
}
```

Use cases: Reduces redundancy in template functions where the return type is complex or depends on template parameters. However, it can reduce readability if overused.

C++14 Language and Library Enhancements

Theory

📌 Generic Lambdas

C++14 allows `auto` in lambda parameters, making them **template-like** without explicit template syntax:

```
auto add = [](auto a, auto b) { return a + b; };
std::cout << add(1, 2) << ", " << add(1.5, 3.5);  // Works for int and double
```

Under the hood, the compiler generates a templated `operator()`:

```
struct {
    template<typename T, typename U>
    auto operator()(T a, U b) const { return a + b; }
} add;
```

This is particularly powerful when combined with perfect forwarding, `enable_if`, or concepts (C++20). Generic lambdas eliminate the need for templated functors in many cases, making code more concise.

Edge case: Type deduction failures can be harder to diagnose than with explicit templates. Using `static_assert` with type traits inside generic lambdas can help enforce constraints.

C++14 Language and Library Enhancements

Theory

📌 Lambda Init-Capture (Move Capture)

C++11 lambdas could only capture by value or reference. C++14 introduces **init-capture**, allowing you to move resources into the lambda:

```
auto ptr = std::make_unique<int>(42);
auto f = [p = std::move(ptr)] { std::cout << *p; };
```

After this capture, `ptr` is left in a moved-from state (typically `nullptr` for `unique_ptr`). Init-capture is essential for:
- **Threading**: Moving unique resources into thread callbacks
- **Async operations**: Capturing ownership for `std::async` or futures
- **RAII closures**: Transferring ownership to a lambda that outlives the current scope

For `shared_ptr`, move capture transfers ownership but doesn't null the original if other copies exist (reference count decreases by one):

```
auto sp = std::make_shared<int>(100);
auto lam = [p = std::move(sp)]() { return *p; };
// sp is now nullptr, lam's p owns the int
```

C++14 Language and Library Enhancements

Theory

📌 decltype(auto) for Exact Type Deduction

While `auto` strips references and const qualifiers, `decltype(auto)` preserves the **exact type** of the expression, including references:

```
int x = 10;
auto f1() { return (x); }           // Returns int (copy)
decltype(auto) f2() { return (x); } // Returns int& (reference)
```

The parentheses around `(x)` make it an lvalue expression, so `decltype((x))` is `int&`. This is critical when perfect forwarding return types or implementing proxy classes. **Warning**: Using `decltype(auto)` with `return (local_var);` creates a dangling reference:

```
decltype(auto) danger() {
    int local = 5;
    return (local);  // Returns int& to destroyed local → UB
}
```

The rule of thumb: use `decltype(auto)` when you need to preserve reference semantics (e.g., forwarding), but be careful with local variables.

C++14 Language and Library Enhancements

Theory

📌 Relaxed constexpr Functions

C++11 `constexpr` functions were extremely limited: single return statement, no local variables, no loops. C++14 relaxed these restrictions dramatically:

C++11 constexpr (restrictive):

```
constexpr int fact(int n) {
    return (n <= 1) ? 1 : n * fact(n - 1);  // Only recursion or ternary
}
```

C++14 constexpr (relaxed):

```
constexpr int fact(int n) {
    int result = 1;
    for (int i = 2; i <= n; ++i)  // ✅ Loops allowed
        result *= i;
    return result;
}
```

C++14 allows:
- Multiple statements
- Local variables
- Loops (for, while)
- Conditional branches (if-else, switch)

This makes `constexpr` practical for real algorithms. Combined with `constexpr` variables, you can compute complex values at compile time, reducing runtime overhead in performance-critical code.

C++14 Language and Library Enhancements

Theory

📌 STL Enhancements

std::make_unique: Finally addresses the missing piece from C++11. Creating `unique_ptr` with `new` was verbose and exception-unsafe:

```
// C++11 (verbose, potential leak if constructor throws)
auto p = std::unique_ptr<Widget>(new Widget(args));

// C++14 (clean, exception-safe)
auto p = std::make_unique<Widget>(args);
```

std::exchange: Replaces a value and returns the old value atomically. Extremely useful in move constructors/assignment:

```
int a = 10;
int old = std::exchange(a, 20);  // a = 20, old = 10
```

std::integer_sequence / std::index_sequence: Enables compile-time index generation for tuple unpacking and variadic expansion:

```
template<typename Tuple, std::size_t... Is>
void print_tuple(const Tuple& t, std::index_sequence<Is...>) {
    ((std::cout << std::get<Is>(t) << " "), ...);  // C++17 fold, but indices from C++14
}
```

std::get(tuple): Access tuple by type (not just index), but only if the type appears once:

```
std::tuple<int, double, char> t{1, 2.5, 'a'};
double d = std::get<double>(t);  // ✅ OK, double appears once
// std::get<int> would fail if int appeared twice
```

C++14 Language and Library Enhancements

Theory

📌 Variable Templates

C++14 introduces **template variables**, allowing templates to define variables directly:

```
template<typename T>
constexpr T pi = T(3.1415926535);

double d = pi<double>;
float f = pi<float>;
```

Before C++14, you'd need a static constexpr member in a template struct. Variable templates are cleaner and more intuitive. They're particularly useful with type traits:

```
template<typename T>
constexpr bool is_numeric = std::is_arithmetic<T>::value;

static_assert(is_numeric<int>, "Expected numeric type");
```

C++14 Language and Library Enhancements

Theory

📌 Syntax Conveniences

Binary literals: `0b` prefix for binary representation:

```
int mask = 0b11110000;  // 240 in decimal
```

Digit separators: Single quotes `'` for readability:

```
int million = 1'000'000;
long long big = 9'223'372'036'854'775'807;  // LLONG_MAX
int binary = 0b1010'1010;  // Works with binary too
```

[[deprecated]] attribute: Mark functions/types as deprecated:

```
[[deprecated("Use newFunc instead")]]
void oldFunc();

oldFunc();  // Compiler warning
```

C++14 Language and Library Enhancements

Edge Cases

📌 1. Return Type Deduction with Multiple Return Paths

```
auto tricky(bool b) {
    if (b) return 1;      // int
    else return 1.5;      // double
}  // ❌ ERROR: Conflicting deduced types
```

All return statements must deduce to the **same type**. The compiler doesn't try to find a common type. This is intentional to prevent surprises. If you need different return types, use explicit return type or `std::variant` (C++17+).

Workaround:

```
double tricky(bool b) {  // ✅ Explicit return type
    if (b) return 1;     // Implicitly converted to double
    else return 1.5;
}
```

Key takeaway: Return type deduction is all-or-nothing; be cautious with conditional returns.

C++14 Language and Library Enhancements

Edge Cases

📌 2. decltype(auto) and Parentheses Matter

```
int x = 10;

auto f1() { return x; }    // Returns int (copy)
auto f2() { return (x); }  // Returns int (copy) - parens ignored for auto

decltype(auto) g1() { return x; }    // Returns int (copy)
decltype(auto) g2() { return (x); }  // Returns int& (reference!) - parens matter
```

With `decltype(auto)`, parentheses change the deduced type:
- `decltype(x)` is `int` (identifier)
- `decltype((x))` is `int&` (expression)

This can lead to **dangling references** if used with local variables:

```
decltype(auto) danger() {
    int local = 5;
    return (local);  // ❌ Returns int& to local → UB when accessed
}
```

Key takeaway: Be extremely careful with `decltype(auto)` and parentheses; use it primarily for perfect forwarding, not with local variables.

C++14 Language and Library Enhancements

Edge Cases

📌 3. Generic Lambdas and Type Deduction Failures

```
auto lambda = [](auto x) {
    return x + x;
};

// lambda({1, 2, 3});  // ❌ ERROR: Can't deduce type for braced-init-list
```

Braced initializer lists have no deducible type in template contexts. You must explicitly specify the type:

```
auto lambda = [](std::initializer_list<int> il) {
    int sum = 0;
    for (int v : il) sum += v;
    return sum;
};
lambda({1, 2, 3});  // ✅ OK
```

Also, generic lambdas with `static_assert` or `enable_if` can provide better compile-time error messages:

```
auto only_numeric = [](auto val) {
    static_assert(std::is_arithmetic<decltype(val)>::value, "Numeric required");
    return val * 2;
};

only_numeric(10);    // ✅ OK
// only_numeric("hi");  // ❌ Compile error with clear message
```

Key takeaway: Generic lambdas don't support braced-init-list deduction; add type constraints for better diagnostics.

C++14 Language and Library Enhancements

Edge Cases

📌 4. Init-Capture and Reference Count with shared_ptr

```
auto sp = std::make_shared<int>(100);
std::cout << sp.use_count() << "\n";  // 1

auto lam = [p = std::move(sp)]() { return *p; };
std::cout << sp.use_count() << "\n";  // 0 (sp is nullptr)
std::cout << (sp == nullptr) << "\n";  // 1 (true)
```

Moving a `shared_ptr` transfers ownership to the lambda. Unlike with `unique_ptr` which always becomes nullptr, `shared_ptr` behavior depends on whether other copies exist. If you copy into the lambda instead:

```
auto sp = std::make_shared<int>(100);
auto lam = [p = sp]() { return *p; };  // Copy, not move
std::cout << sp.use_count() << "\n";  // 2 (shared between sp and lambda)
```

Key takeaway: Understand move vs copy semantics with init-capture; unique_ptr forces move-only, shared_ptr allows both.

C++14 Language and Library Enhancements

Edge Cases

📌 5. Variable Templates and ODR (One Definition Rule)

```
// header.h
template<typename T>
constexpr T pi = T(3.1415926535);
```

Each translation unit that includes this header gets its own instantiation of `pi<double>`, but they all have the same address due to linkage rules. However, if `pi` were non-const or non-constexpr, you'd violate ODR:

```
template<typename T>
T mutable_value = T(0);  // ❌ Each TU gets its own copy → linker error if used across TUs
```

Key takeaway: Variable templates should be const or constexpr to avoid ODR violations in headers; use inline variables (C++17) for mutable shared state.

C++14 Language and Library Enhancements

Code Examples

📌 1. Return Type Deduction in Template Functions

Return type deduction eliminates the verbose `typename Container::value_type` while maintaining full generic capability. This is particularly useful in complex template metaprogramming where return types depend on multiple template parameters.

```
#include <iostream>
#include <vector>

// ✅ Without return type deduction (C++11)
template<typename Container>
typename Container::value_type get_first_old(const Container& c) {
    return c.empty() ? typename Container::value_type{} : c.front();
}

// ✅ With return type deduction (C++14)
template<typename Container>
auto get_first(const Container& c) {
    return c.empty() ? typename Container::value_type{} : c.front();
}

int main() {
    std::vector<int> v = {1, 2, 3};
    std::cout << get_first(v) << "\n";  // 1

    std::vector<std::string> vs = {"hello", "world"};
    std::cout << get_first(vs) << "\n";  // "hello"
}
```

C++14 Language and Library Enhancements

Code Examples

📌 2. Generic Lambdas with Perfect Forwarding

This pattern is common in middleware layers that wrap function calls with logging, timing, or error handling. The generic lambda with perfect forwarding preserves lvalue/rvalue reference semantics, allowing modification of passed references.

```
#include <iostream>
#include <memory>
#include <utility>

// Generic lambda that perfectly forwards arguments
auto invoke_with_logging = [](auto&& func, auto&&... args) {
    std::cout << "Calling function with " << sizeof...(args) << " args\n";
    return std::forward<decltype(func)>(func)(std::forward<decltype(args)>(args)...);
};

void sensor_process(int& value) {
    value *= 2;
    std::cout << "Processed: " << value << "\n";
}

int main() {
    int sensor_data = 42;
    invoke_with_logging(sensor_process, sensor_data);
    std::cout << "Final value: " << sensor_data << "\n";  // 84

    // Also works with lambdas
    invoke_with_logging([](int x, int y) { return x + y; }, 10, 20);
}
```

C++14 Language and Library Enhancements

Code Examples

📌 3. Lambda Move-Capture in Threading

In autonomous driving systems, sensor data processing often happens asynchronously. Move-capture allows transferring ownership of large data structures into worker threads without copying, improving performance and memory efficiency.

```
#include <iostream>
#include <thread>
#include <memory>
#include <chrono>

struct SensorData {
    int id;
    std::vector<double> readings;
    SensorData(int i) : id(i), readings(1000, 42.0) {}  // Large data
};

void process_async(int sensor_id) {
    auto data = std::make_unique<SensorData>(sensor_id);

    // ✅ Move unique_ptr into thread lambda
    std::thread worker([d = std::move(data)] {
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
        std::cout << "Sensor " << d->id << " processed " << d->readings.size() << " readings\n";
    });

    // data is now nullptr - ownership transferred to thread
    worker.join();
}

int main() {
    process_async(1);
    process_async(2);
}
```

C++14 Language and Library Enhancements

Code Examples

📌 4. decltype(auto) for Perfect Return Forwarding

This demonstrates the power of `decltype(auto)` in generic code that needs to preserve reference semantics. It's particularly important with proxy types like `vector<bool>::reference`.

```
#include <iostream>
#include <vector>

// ✅ Perfect return forwarding - preserves reference
template<typename Container>
decltype(auto) get_element(Container& c, size_t index) {
    return c[index];  // Returns Container::reference (which may be T& or proxy type)
}

// ❌ Without decltype(auto) - always returns copy
template<typename Container>
auto get_element_copy(Container& c, size_t index) {
    return c[index];  // Returns Container::value_type (copy)
}

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5};

    // With decltype(auto) - can modify through returned reference
    get_element(v, 0) = 100;
    std::cout << v[0] << "\n";  // 100

    // With auto - returns copy, can't modify original
    // get_element_copy(v, 1) = 200;  // ❌ Error: can't assign to temporary

    std::vector<bool> vb = {true, false, true};
    // decltype(auto) preserves proxy type from vector<bool>
    get_element(vb, 0) = false;
    std::cout << vb[0] << "\n";  // 0 (false)
}
```

C++14 Language and Library Enhancements

Code Examples

📌 5. Relaxed constexpr for Compile-Time Algorithms

In embedded or real-time systems, precomputing lookup tables at compile time eliminates runtime overhead. C++14's relaxed constexpr makes this practical for complex algorithms, not just simple calculations.

```
#include <iostream>
#include <array>

// ✅ C++14: Complex algorithm at compile time
constexpr int fibonacci(int n) {
    int a = 0, b = 1;
    for (int i = 0; i < n; ++i) {
        int temp = a;
        a = b;
        b = temp + b;
    }
    return a;
}

// Compile-time array generation
constexpr std::array<int, 10> generate_fib_array() {
    std::array<int, 10> arr = {};
    for (size_t i = 0; i < arr.size(); ++i) {
        arr[i] = fibonacci(i);
    }
    return arr;
}

int main() {
    constexpr auto fib_array = generate_fib_array();  // ✅ Computed at compile time
    for (int val : fib_array) {
        std::cout << val << " ";
    }
    // Output: 0 1 1 2 3 5 8 13 21 34
}
```

C++14 Language and Library Enhancements

Code Examples

📌 6. std::exchange in Move Operations (1/2)

`std::exchange` is cleaner than the traditional "swap and nullify" pattern in move operations. It's a single expression that replaces the old value and returns it, making move constructors more concise and less error-prone.

```
#include <iostream>
#include <utility>
#include <memory>

class ResourceHandle {
    int* data;
public:
    ResourceHandle(int val = 0) : data(new int(val)) {
        std::cout << "Constructed with " << *data << "\n";
    }

    ~ResourceHandle() {
        if (data) {
            std::cout << "Destroying " << *data << "\n";
            delete data;
        }
    }

    // Move constructor using std::exchange
    ResourceHandle(ResourceHandle&& other) noexcept
        : data(std::exchange(other.data, nullptr)) {  // ✅ Atomic swap
        std::cout << "Moved, source reset to nullptr\n";
    }

    // Move assignment using std::exchange
    ResourceHandle& operator=(ResourceHandle&& other) noexcept {
        if (this != &other) {
            delete data;  // Clean up existing resource
            data = std::exchange(other.data, nullptr);
            std::cout << "Move-assigned\n";
        }
        return *this;
    }

    ResourceHandle(const ResourceHandle&) = delete;
    ResourceHandle& operator=(const ResourceHandle&) = delete;
// ...continues on next slide
```

C++14 Language and Library Enhancements

Code Examples

📌 6. std::exchange in Move Operations (2/2)

[Code continues across slides]

```
// ...continued from previous slide

    int value() const { return data ? *data : -1; }
};

int main() {
    ResourceHandle h1(42);
    ResourceHandle h2 = std::move(h1);
    std::cout << "h1 value: " << h1.value() << "\n";  // -1 (nullptr)
    std::cout << "h2 value: " << h2.value() << "\n";  // 42
}
```

C++14 Language and Library Enhancements

Code Examples

📌 7. Variable Templates with Type Traits

Variable templates combined with SFINAE provide cleaner, more readable template metaprogramming compared to C++11's `::value` verbosity. This is especially beneficial in large codebases with heavy template usage.

```
#include <iostream>
#include <type_traits>

// Variable templates for cleaner metaprogramming
template<typename T>
constexpr bool is_numeric = std::is_arithmetic<T>::value;

template<typename T>
constexpr bool is_pointer_type = std::is_pointer<T>::value;

// SFINAE with variable templates
template<typename T>
std::enable_if_t<is_numeric<T>> process_value(T val) {
    std::cout << "Processing numeric: " << val * 2 << "\n";
}

template<typename T>
std::enable_if_t<is_pointer_type<T>> process_value(T ptr) {
    if (ptr) std::cout << "Processing pointer: " << *ptr << "\n";
    else std::cout << "Null pointer\n";
}

int main() {
    process_value(42);          // Numeric overload
    process_value(3.14);        // Numeric overload

    int x = 100;
    process_value(&x);          // Pointer overload
    process_value<int*>(nullptr);  // Pointer overload
}
```

What You Learned

Key Takeaways

- ✓ Return Type Deduction for Normal Functions
- ✓ Generic Lambdas
- ✓ Lambda Init-Capture (Move Capture)
- ✓ decltype(auto) for Exact Type Deduction

→ *Swipe to learn* →